Cao Nguyễn Hữu Thuận (1)
N25DECE069
E25CQCE02-N

Problem 1 Singly Linked list - In Place ~~Reversal~~ Reversal & Pointer Tracing.

Given a singly linked list:

head → [10] → [25] → [40] → [15] → NIL

Task 1: Write a function / algorithm in pseudocode to reverse the singly linked list including three pointers (prev, curr, next) to complete your functio

Task 2: Draw a diagram or step-by-step pointer trace table showing the positions and values of the three pointers (prev, curr, next) after each interation until the list is fully reversed.

Ans

Task 1: Pseudocode

```
Plaintext.
function reverseLinkedList (head):
    prev = NILLL
    curr = head
    While (curr is not NULL)
    next = curr.next          // Lưu lại node tiếp theo
    curr.next = prev          // Đảo chiều liên kết
    prev = curr               // Tiến prev lên
    curr = next               // Tiến curr lên
    return head               // prev
```

1.25 Task2

Iteration	prev	curr	next	Trạng thái danh sách liên kết (head vẫn ở 10
0 Start	NULL	10	–	10→25→40→15→NULL
After round 1	10	25	25	10→NULL
After round 2	25	40	40	25→10→NULL
After round 3	40	15	15	40→25→10→NU
After round 4	15	NULL	NULL	15→10→25→10→NULL

End: Because curr == NULL, Assign head = prev (Note 15)

New list: head → [15] → [40] → [25] → [10]

2.5 Problem 2. Circular Linked list - Insertathead → NULL

Given a singly circular linked list: head → [10] → [20] → [30] → ...

Task 1: Write pseudocode for the function InsertAtHead(head, newValue) to insert a new node containing newValue at the front of the circular linked list while maintaining its circular structure

Task2:

BÚT CẦU

Scanned with CamScanner

1,25 Task 1.

```
function InsertAtHead(head, newValue):
    newNode = createNode(newValue)
    // Trường hợp danh sách rỗng
    if head is NULL:
        newNode.next = newNode
        return newNode
    // Duyệt tìm nút cuối cùng (tail)
    tail = head
    while (tail.next != head)
        tail = tail.next
    // Chèn nút mới vào trước head và cập nhật liên kết
    newNode.next = head
    tail.next = newNode
    head = newNode   // Cập nhật head mới
    return head
```

1,25 Task 2:

Initial list: [10] → [20] → [30] → (10)

1. Create a new node: newNode contains the value 5
2. Find tail: Traverse through nodes 10 → 20 → 30. Notice that 30.next == head (which is node 10)

Stop with tail = 30

HL BÚT CÂU

Scanned with CamScanner

3. Update pointers

- newNode.next = head → 5. next points to 10
- tail.next = newNode → 30. next points to 5.
- head = newNode → new head is node 5.

→ Result: head → [5] → [10] → [20] → [30] → (points back to 5).

1,75 Problem 3. Doubly linked list - Insert Before Target & Edge Cases

Given a doubly linked list containing integers.

Task 1. Write pseudocode for InsertBefore(head, target, newValue) to insert a new node containing newValue immediately before the first occurrence of target.

Task 2: List the exact order of pointer assignments required and briefly justify why this specific sequence avoids memory leaks or broken links.

Task 3: Explicitly handle the following two edge cases in code / pseudocode.

1. target is located at the head node (head)
2. target does not exist in the list

HL BÚT CẦU

Scanned with CamScanner

Task 1 & Task 3

curr = head

// Tìm nút chứa giá trị target

while (curr is not NULL and curr.data != target)

curr = curr.next

// Trường hợp 2 : Target không tồn tại trong danh sách

if curr IS NULL:

printf ("Target not found")

return head

// Tạo nút mới

newNode = createNode(newValue)

// Trường hợp 3 : Target nằm ở đầu nút head

if curr = head

newNode.next = head

head.prev = newNode

head = NewNode

return head.

// Trường hợp 4. Target nằm ở giữa danh sách

newNode.prev = curr.prev

newNode.next = curr

curr.prev.next = newNode

curr.prev = newNode.

return head.

Quang Tâm

Scanned with CamScanner

1.5 Problem 4

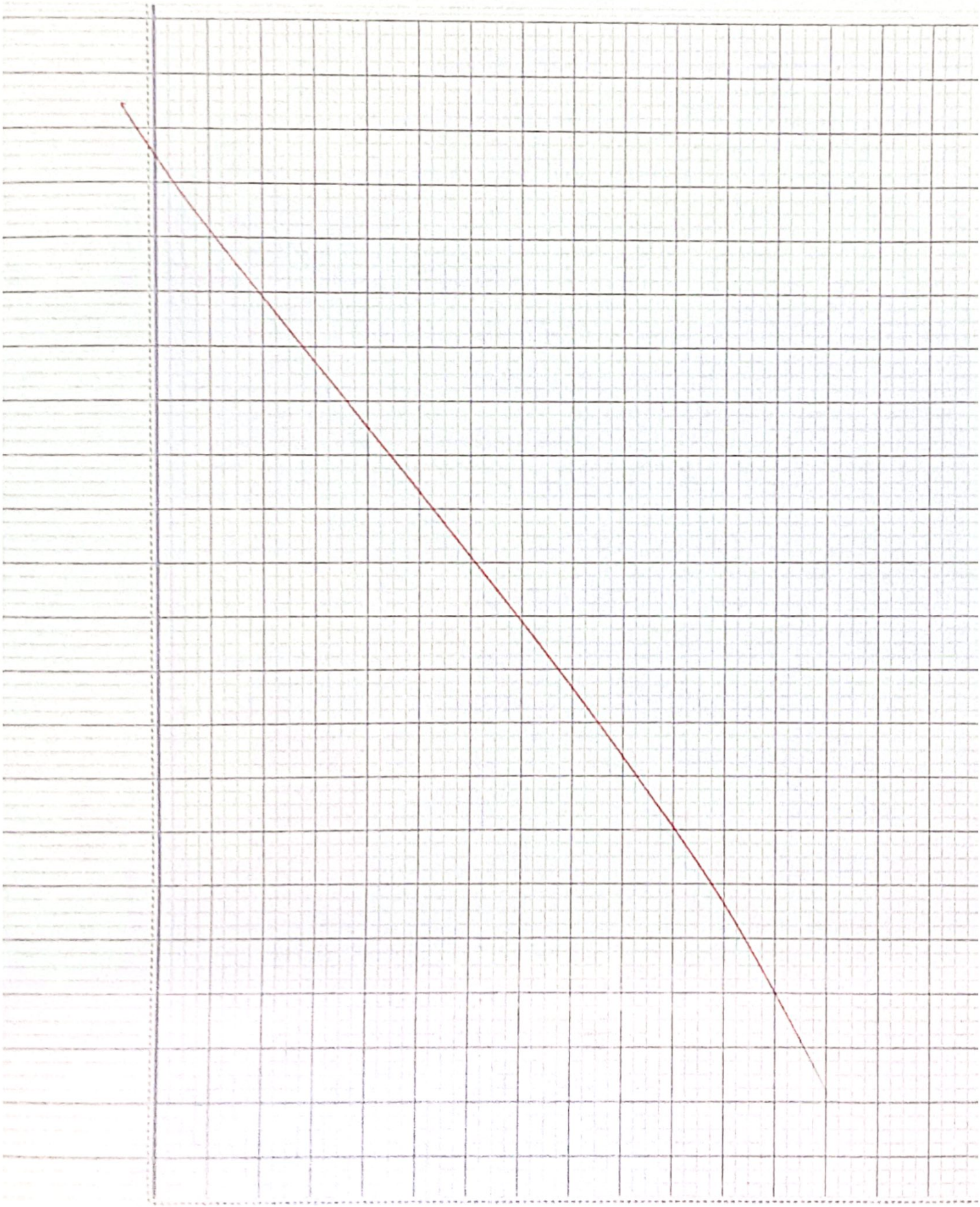
Given a singly linked list sorted in ascending order
head → [2] → [5] → [8] → NULL.

Ans:

```
function InsertInOrder(head, newValue):
    newNode = createNode(newValue)
// Trường hợp 1: Danh sách rỗng hoặc newValue nhỏ hơn head.
    if head (is NULL or newValue < head.data):
        newNode.next = head.
        head = newNode
        return head.
// Tìm vị trí thích hợp để chèn.
    curr = head.
while (curr.next is not NULL and curr.next.data < newValue):
        curr = curr.next
// Chèn newNode vào sau curr
    newNode.next = curr.next.
    curr.next = newNode.
    return head.
```

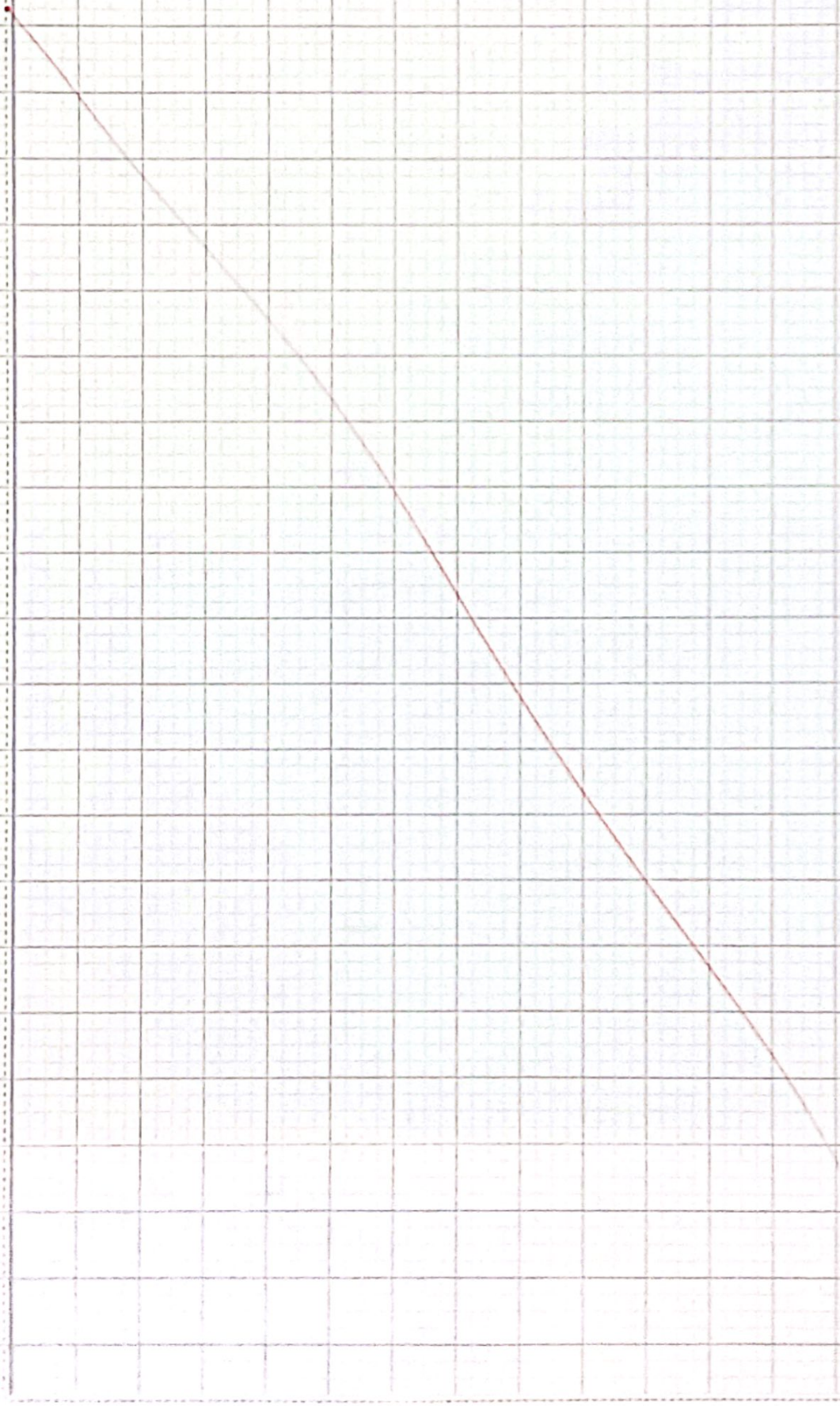

Quang Tâm

Scanned with CamScanner



Quang Tâm

Scanned with CamScanner



Quang Tâm

Scanned with CamScanner